

FRED TALKS

16th May 2026

CONTENTS

Semantic Search Without Embeddings

DOUG TURNBULL · 3143 WORDS

Where the goblins came from | OpenAI

OPENAI · 1128 WORDS

[Daily article] May 16: Erik Campbell (Final Destination)

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 327 WORDS

Semantic Search Without Embeddings

DOUG TURNBULL · 08 JAN 2026 · [SOURCE](#)

When I need to stay warm, I search for “long johns” or “long underwear”. But modern outdoors clothing stores label this a “base layer”. For a waterproof jacket I search for “slickers” or a “ski jacket”, not realizing what I should search for is a “shell.”

Despite my outdated terminology, search still works. I’m somehow understood and shown the right content.

We call this **semantic search**. When you hear that, you might think embeddings. Today I want to stretch your thinking beyond. Fantasize and semanticize.

Some teams wrongly assume semantic search equates to only “vector search”. You have more options, some that might better align to your domain and existing organizational skills. LLMs simplify classic ways of organizing search queries and information.

Let’s contrast different approaches, and maybe you’ll see there’s another approach that might be a better fit for you.

Semantic search - what exactly is it?

In semantic search, content and query map to a ***shared representation***. This space has a ***similarity*** function that scores similar items higher than less similar.

Let’s walk through an example.

A user searching “round red fruit that grows on trees” probably wants an apple.

Humans know this - search engines don’t. So we need to help.

We map the query to three properties: [round, red, fruit]

In our search index we’ve also mapped three items into that space:

An **apple**: [round, red, fruit]

An **orange**: [round, orange, fruit]

A **baseball**: [round, white, ball]

Now we have a shared *representation*: a three tuple of (shape, color, item type).

How do we measure similarity? For each property (shape, color, item type) we could score a “1” if they’re equal, otherwise a 0. Then add to get a total score:

Similarity(query, apple) = 1 + 1 + 1 = 3

Similarity(query, orange) = 1 + 0 + 1 = 2

Similarity(query, baseball) = 1 + 0 + 0 = 1

So the user sees search results

1. Apple (Score 3)
2. Orange (Score 2)
3. Baseball (Score 1)

Ok, so a fairly silly method of semantic search. It ignores properties that matter (size, for example). It requires us to comprehensively tag everything. Still many teams try to tag + synonym their way to better search, with middling results.

Embeddings for semantic search

The default answer to the “semantic search” question is embeddings. The Internet is littered with documentation for [how vector search works](#). I’ll do the speedrun here ([references here](#)).

We can learn a representation, an embedding, for apple and orange so that `similarity(apple, orange) > similarity(apple, baseball)`.

We learn an embedding from training data. We might declare when items get clicked for the same query, they are similar. We notice both “apple” and “orange” get clicked for query “fruit that grows on trees” at high frequency:

[fruit that grows on trees 🔍]

1. 🍏 (clicked)
2. 🥕
3. 🍊 (clicked)

We start with a random vector for each. A list of random floating points numbers like `[0.01, -0.1, 0.05]`. Then we train, in this super-secret-ultra-high-tech algorithm

```
for session in sessions:
    together = []
    for lhs in session: # ie the apple
        for rhs in session: # ie the orange
            if lhs != rhs:
                nudge_closer(lhs, rhs) # ie nudge a
```

(Not shown: nudging away the negative, dissimilar items)

After training, for many queries+sessions, maybe apple becomes `[0.1, -0.2, 0.05]` while orange becomes a similar vector: `[0.09, -0.21, 0.04]`. We measure the similarity with cosine similarity, euclidean distance, and others. (We can replace “shared search session clicks” with any other notions of “similar”: occurs in the same text passages, both occur in shopping carts, etc).

Learning an embedding per item presents problems. For example, maybe we just added strawberry 🍓 to our product listings. With no training data, the vector `strawberry_vector = embeddings['strawberry']` initializes to random garbage. It won't have a meaningful value until we've shown it to many users.

We'd prefer to compute embeddings from *features* of the item. Then strawberry 🍓 could get an embedding right away. IE instead of `strawberry_embedding = embeddings['strawberry']` we can do `strawberry_embedding = f(heart_shaped, red, fruit)`. We build an *encoder*. Given the item's attributes we compute embeddings that produce the expected similarity. Embedding models downloaded from hugging face specialize in encoding specific type/domain of content for a specific task (search, etc).

We create encoders for all kinds of features: ordinal values (like a list of shapes, colors), the raw title / description text itself, images and so on. We can also train a two-tower model: one encoder for queries - ie “round fruit that goes on trees” - and another for documents.

Like other machine learning models we tradeoff precision for generality. We need the right features, we need enough training data for all the variations in those features (curse of dimensionality) and so on.

The missing semantic search piece of vector retrieval

One pain point comes up right away with embeddings: lack of *matching*.

How would users actually react to these search results?

[fruit that grows on trees 🔍]

1. 🍏 Apple
2. 🍊 Orange
3. ⚾ Baseball.

In my experience, negatively.

It may have the perfect ranking. But search should exclude items outside what the user intended. They find fruit acceptable. But not baseballs.

So earlier I lied. What users actually want from semantic search:

- **Representation** - some space both queries and content can share representation
- **A similarity function** - A way of measuring how near / far items in the shared representation are from each other
- **Match criteria (new!)** - A system where an item can be said to be “match” or not for the query in the semantic space. It is the thing, or its not the thing.

In other words, I gave an incomplete **semantic search definition**. Embeddings thrive on the first 2, but don’t do a great job at including / excluding results.

Understanding users (and their queries) matters a great deal in search. That requires including / excluding appropriate items based on terminology and categories familiar to users.

You might think “I’ll just find a cutoff in my embedding similarity”. Sadly, there’s not a magical “at 0.8 similarity it’s not a match”. In some domains / queries one threshold works. In another, a very different threshold. Further, thresholding becomes tougher when users give multiple criteria. An embedding model might rank blood oranges above green apples for query “red apple”. What matters more, the color or item type in the similarity?

Thresholds help exclude some obviously irrelevant results and improve performance, but they don’t use domain-specific criteria, in the user’s language, to include / exclude results.

The other semantic search approach (hierarchical managed taxonomies)

One system tries to solve all three problems: *Managed vocabularies or taxonomies*.

Taxonomies map queries + content to a hierarchy of concepts, using the language of the domain. Think a neatly organized directory tree. Or the Dewey Decimal System

Here's an example from the Wayfair WANDS furniture dataset of their category tree; Where you'd categorize "novelty rocking horses" apparently:

Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses

We can take a query for **hobby horse**. A human, curating a bunch of query rules, could manually map this to a node:

Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses

Assuming the corpus categorizes products correctly, we can search, filter, boost, etc using this.

We have a *representation* (the category tree), do we have a similarity function?

Yes, we can rank as follows:

1. Direct matches Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses
2. Parents / siblings. Baby & Kids / Toddler & Kids Playroom / Indoor Play / Dollhouses
3. Those rank higher than grandparents / cousins (`Baby & Kids / Toddler & Kids Playroom / Beanbag Chairs / ...`)

We can define meaningful match criteria too. We include shared parent categories (ie siblings). But exclude anything beyond cousins / shared grandparents as needed.

Many teams already maintain systems for organizing and cataloging information. For many domains, a good taxonomy can be a moat. Especially for technical domains (legal, medical, fashion, finance) where precise categorization really matters. Existing taxonomies already exists, and statistical fuzziness of embeddings can create huge downside.

In the past, the management of a taxonomy would be massive. It might require complex rules, mapping of query phrases to taxonomy nodes, and a team of labelers and experts to keep everything straight. This has long been an appeal of embeddings.

But LLMs sneakily supercharge these old school ways of organizing information. Given a taxonomy, LLMs can easily + cheaply classify products and queries. It's more approachable to "fine tune". AI augmented labeling can simplify management of knowledge making it more approachable.

Semantic similarity doesn't need a vector index

Above I described a similarity function above (direct matches, then parents, then grandparents). With some creative tokenization, this similarity can happen in a normal BM25 index.

Consider our query for **hobby horse**. We know this maps to this node:

Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses

How do we score content in such a way so that direct matches rank higher than siblings, and so on?

We need is a hierarchical tokenizer. A function that produces the full path of every parent directory:

```
In: hierarchical_tokenizer("Baby & Kids / Toddler & Kids Playroom / I
Out: ['Baby & Kids',
      'Baby & Kids / Toddler & Kids Playroom',
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play',
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking
```

If you index using this tokenizer, you'll recreate scoring needed for the direct > parent > grandparent similarity function I described earlier.

The key reason: BM25 rewards a rare match over common match.

Root nodes (ancestors) have higher document frequency, they're common. Common matches score lower in BM25.

That's because every item labeled `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses` tokenized hierarchically has a `Baby & Kids` term. So does a document labeled `Baby & Kids / Kids Furniture ... / Beanbag Chairs`. `Baby & Kids` happens all over the place - maybe like 10% of the index.

Child nodes have low document frequency, they're rare. In BM25, rare, more specific matches outrank common ones.

A product labeled `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses / Traditional Rocking Horses` when tokenized will include the full path in the token. There might be 10 products in the entire index in this exact category. So that rareness - that high specificity - shoots this narrower, closer match to the top.

To do a deeper walkthrough

When I come along and search for

```
In: hierarchical_tokenizer(  
    "Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking H  
    )
```

```
Out: ['Baby & Kids',  
      'Baby & Kids / Toddler & Kids Playroom',  
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play',  
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking  
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking
```

I'm searching for five tokens now, OR'd (really summed):

```
"Baby & Kids" OR  
"Baby & Kids / Toddler & Kids Playroom" OR  
"Baby & Kids / Toddler & Kids Playroom / Indoor Play" OR  
"Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses  
"Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses
```

Each token of my query has a job. It matches different levels of specificity, and the closer we get the full query classification tree, the higher the score.

		Score	Example of matches
Search Term	Job	(due to doc freq)	
"Baby & Kids"		Lowest	

	Matches highest level category		Baby & Kids / Toddler & Kids Bedroom Furniture / Kids Headboards
"Baby & Kids / Toddler & Kids Playroom"	Matches next most specific category Getting to "play" stuff for kids, better than just kid stuff. 👍	Low	Baby & Kids / Toddler & Kids Playroom / Playroom Furniture / Baby Gyms & Playmats / Folding Baby Gyms & Playmats
...	...	...	...
Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses	Matches a very, narrow specific category. Horses! 🐎 Even if not "Novelty Rocking Horses" its a defensible "match" to the user.	High	Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses / Traditional Rocking Horses

While we keep things in horse → toys → playroom → kids order, we haven't solved the 3rd problem. Limiting matches.

That one's easy. At query time, you can decide how much to limit retrieval. Perhaps you only query the direct node and one parent:

"Baby & Kids / ... / Rocking Horses" OR
 "Baby & Kids / ... / Rocking Horses / Novelty Rocking Horses"

You can tighten / relax in categories that matter to users, *semantically*, up a layer to still stay within the realm of "kid play".

If the taxonomy makes sense to users, if it reflects search behaviors (big ifs), they'll understand why you include / exclude certain results.

Importantly, this becomes just ONE component of the ranking / filtering. We can still have other ingredients we can use:

- Keyword matches (for exact mention of properties in titles + description)
- Embedding search (perhaps within the category matches)
- Taxonomies for other entities (colors, materials, etc)

Building a taxonomy: start simple and evolve

Building and maintaining a good taxonomy is no joke. That's why there's a field of taxonomists you can hire. And I don't want to elide the tradeoffs with embeddings here. It's work!

A taxonomy doesn't need to be as deep / complex as what I've laid out. You can start simple. Ask an LLM. Here's one with color.

Take the 7 primary colors. Then nested under each, create subtypes of each color.
Generate a list like a directory trees, in the form PRIMARY / SECONDARY

RED / Crimson

RED / Scarlet

RED / Burgundy

RED / Maroon

RED / Brick Red

RED / Cherry

ORANGE / Tangerine

ORANGE / Amber

ORANGE / Coral

ORANGE / Burnt Orange

ORANGE / Peach

ORANGE / Apricot

...

It's a start.

Generally start simple. Broad, high level, top level categories like Furniture or Baby & Kids. Then eventually you realize this category has become too large and too diverse. You might see a natural in the content split:

Baby & Kids / Toddler & Kids Playroom

Baby & Kids / Toddler & Kids Bedroom Furniture

Eventually you get to a point where the distinction between Novelty Rocking Horse and Classic Rocking Horse matters for some reason.

You may see a flaw: an assumption each product sits within a single category. That categories are mutually exclusive. Reality, of course, is messy: some products may sit between categories. You can imagine a small goofy looking couch that could reasonably tagged under Baby &

Kids / Toddler & Kids Bedroom Furniture but also Living Room / Loveseats. In Fashion, some activewear might also be underwear. And what about athleisure?

One solution could be to double categorize an item. Give it both classifications - with some weight preferring one categorization over another.

A final concern - there's a big difference between the official pure way of classifying furniture and how a search user might categorize the products. The exacting, furniture-expert taxonomy might have very sophisticated (and accurate) ways to classify furniture. A messier classification that matches how users think about things might be more useful in the end.

Classifying into taxonomies has never been easier

Building taxonomy classifiers with LLMs - something I've built an [entire class for](#), and could be a post on its own. But lets do a quick walkthrough of some simpler approaches.

Embeddings actually help us here. The sweet spot of embeddings in search might be building better classifiers, not in direct ranking and retrieval.

Perhaps our products fit in a few thousand unique categories. Imagine we encode an embedding each of these in a small in-memory array of vectors. We'll use it to lookup categories for our queries.

Now take a query "hobby horse". Encode the query into an embedding, and find the most similar category embedding. Maybe this retrieves Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses.

Not accurate enough? Let's take it up a notch.

We could ask an LLM to hallucinate a fake label before categorizing. Ask an LLM the following:

Be creative and hallucinate a set of classifications for the query below that look like the real classifications.

Product classifications might look like:

```
'Furniture / Living Room Furniture / Coffee Tables & End Tables / C  
'Décor & Pillows / Decorative Pillows & Blankets / Throw Pillows'
```

'Furniture / Bedroom Furniture / Dressers & Chests'
'Outdoor / Outdoor & Patio Furniture / Patio Furniture Sets / Patio
'Home Improvement / Bathroom Remodel & Bathroom Fixtures / Bathroom
'Lighting / Wall Lights / Bathroom Vanity Lighting'
'Kitchen & Tabletop / Kitchen Organization / Food Storage & Caniste
'School Furniture and Supplies / School Furniture / School Chairs &
'Baby & Kids / Toddler & Kids Bedroom Furniture / Kids Beds',

If you feel inspired, return many unique values in a list. Be creative
Cast a wide net with related but diverse categories.

Here's the query to invent categories for:

hobby horse

When you actively want hallucinations, you can get by with very small models. You're not
relying on the LLM for accuracy, just creativity and close-enough language generation.

Now let's say you do this. It gives you a classification like

'Baby & Kids / Toys / Pretend Play & Dress Up / Pretend Play Toys / H

More descriptive, closer to the language actually used in the classifications. More likely to give
back a real, high quality classification.

That's just a taste. The modern landscape for building large multi-label classifiers has gotten
easier than ever thanks to LLMs and embeddings.

Final thoughts

Embeddings are great. But you don't need embeddings to build "semantic search". In fact, it can
be counter productive for problems requiring exacting, precise matching. As I've argued,
embeddings are the wrong lens to think about the problem.

Maybe you don't need to rebuild everything from scratch, but instead leverage what you're good
at. With the addition of LLMs. Don't apologize for living with a more organized, approach to
semantic search.

Enjoy softwaredoug in training course form!

20% off through December 19, 2025 with code ps5



I hope you join me at [Cheat at Search with LLMs](#) to learn how to apply LLMs to search applications. Check out [this post](#) for a sneak preview.

Where the goblins came from | OpenAI

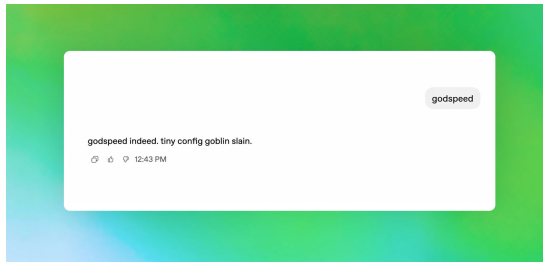
OPENAI · 02 MAY 2026 · [SOURCE](#)

April 29, 2026

[Publication](#)

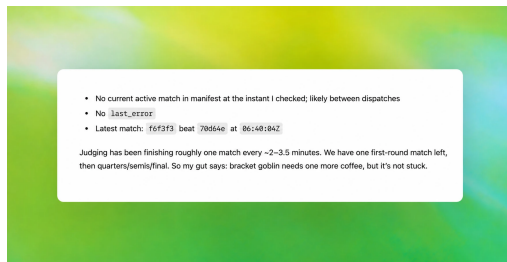
Share

Starting with GPT-5.1, our models began developing a strange habit: they increasingly mentioned goblins, gremlins, and other creatures in their metaphors. Unlike model bugs that show up through a tanking eval or a spiking training metric and point back to a specific change, this one crept in subtly. A single “little goblin” in an answer could be harmless, even charming. Across model generations, though, the habit became hard to miss: the goblins kept multiplying, and we needed to figure out where they came from.

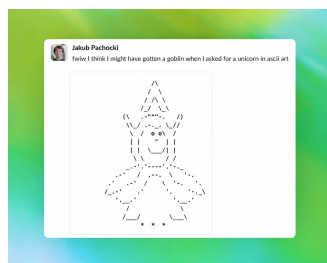


In early testing, GPT-5.5 in Codex showed an odd affinity for goblin metaphors.

The short answer is that model behavior is shaped by many small incentives. In this case, one of those incentives came from training the model for the personality customization feature, in particular the Nerdly personality. We unknowingly gave particularly high rewards for metaphors with creatures. From there, the goblins spread.



The goblins were funny at first, but the increasing number of employee reports became concerning.



An interesting interaction our Chief Scientist had with GPT-5.5.

The first signs of creatures

The first time we clearly saw the pattern was in November, after the GPT-5.1 launch, although it may have started earlier. Users complained about the model being oddly overfamiliar in conversation, which prompted an investigation into specific verbal tics. A safety researcher had experienced a few “goblins” and “gremlins” and asked that they be included in the check. When we looked, use of “goblin” in ChatGPT had risen by 175% after the launch of GPT-5.1, while “gremlin” had risen by 52%.

A measurable small lexical quirk in GPT-5.1.

At the time, the prevalence of goblins did not look especially alarming. A few months later, the goblins came back to haunt us in a much more specific and reproducible form.

Solving the goblin mystery

With GPT-5.4, we and our users noticed an even bigger uptick in references to these creatures. That triggered another internal analysis and surfaced the first connection to the root cause: creature language was especially common in production traffic from users who had selected the “Nerdy” personality. “Nerdy” used the following system prompt, which partially explained the quirkiness:

You are an unapologetically nerdy, playful and wise AI mentor to a human. You are passionately enthusiastic about promoting truth, knowledge, philosophy, the scientific method, and critical thinking. [...] You must undercut pretension through playful use of language. The world is complex and strange, and its strangeness must be acknowledged, analyzed, and enjoyed. Tackle weighty subjects without falling into the trap of self-seriousness. [...]

If the behavior were simply a broad internet trend, we would expect it to spread more evenly. Instead, it was clustered in the part of the system explicitly optimized for a playful, nerdy style. Nerdy accounted for only 2.5% of all ChatGPT responses, but 66.7% of all “goblin” mentions in ChatGPT responses.

The behavior was highly concentrated in the "Nerdy" personality.

Because “goblin” prevalence seemed to increase over our model releases, we had a suspicion that something in our personality instruction-following training was amplifying this.

Codex helped us compare model outputs generated during RL training containing goblin or gremlin with outputs from the same task that did not. One reward signal stood out immediately: the one originally designed to encourage the Nerdy personality was consistently more favorable to the creature-word outputs. Across all datasets in the audit, the Nerdy personality reward showed a clear tendency to score outputs to the same problem with “goblin” or “gremlin” higher than outputs without, with positive uplift in 76.2% of datasets.

That explained why the behavior was boosted with the Nerdy personality prompt, but not why it also appeared without that prompt. To test whether the style was transferring, we tracked mention rates over training both with and without the Nerdy prompt.

As goblin and gremlin mentions increased under the Nerdy personality, they increased by nearly the same relative proportion in samples without it. Taken together, the evidence suggests that the broader behavior emerged through transfer from Nerdy personality training.

The rewards were applied only in the Nerdy condition, but reinforcement learning does not guarantee that learned behaviors stay neatly scoped to the condition that produced them. Once a style tic is rewarded, later training can spread or reinforce it elsewhere, especially if those outputs are reused in supervised fine-tuning or preference data.

That creates a feedback loop:

1. Playful style is rewarded
2. Some rewarded examples contain a distinctive lexical tic.
3. The tic appears more often in rollouts.
4. Model-generated rollouts are used for supervised fine-tuning (SFT).
5. The model gets even more comfortable producing the tic.

A search through GPT-5.5’s SFT data found many datapoints containing “goblin” and “gremlin.” Further investigation revealed a whole family of other odd creatures: raccoons, trolls, ogres, and pigeons were identified as other tic words, while most uses of frog turned out to be legitimate.

One week average of production prevalence of goblins and gremlins. The drop in GPT-5.4 Thinking was a result of retiring the “Nerdy” personality mid-March. GPT-5.5 never launched with the “Nerdy” personality, and showed another increase over GPT-5.4 (even without “Nerdy”).

The end of the goblins

We retired the “Nerdy” personality in March after launching GPT-5.4. In training, we removed the goblin-affine reward signal and filtered training data containing creature-words, making goblins less likely to over-appear or show up in inappropriate contexts. Unfortunately, GPT-5.5 started training before we found the root cause of the goblins. When we began testing GPT-5.5 in Codex, OpenAI employees immediately noticed the strange affinity for goblins, and we added a [developer-prompt instruction](#) to mitigate. Codex is, after all, quite nerdy.

If you want to let the creatures run free in Codex, you can run this command to launch Codex with the goblin-suppressing instructions removed:

Plain Text

```
instructions=$(mktemp /tmp/gpt-5.5-instructions.XXXXXX) && \
jq -r '.models[] | select(.slug=="gpt-5.5") | .base_instructions' \
~/codex/models_cache.json | \
grep -vi 'goblins' > "$instructions" && \
codex -m gpt-5.5 -c "model_instructions_file=\"$instructions\""
```

Why it matters

Depending on who you ask, the goblins are a delightful or annoying quirk of the model. But they are also a powerful example of how reward signals can shape model behavior in unexpected ways, and how models can learn to generalize rewards in certain situations to unrelated ones. Taking the time to understand why a model is behaving in a strange way, and building out ways to investigate those patterns quickly, is an important capability for our research team. This investigation resulted in new tools for the research team to audit model behavior and fix behavior problems at their root.

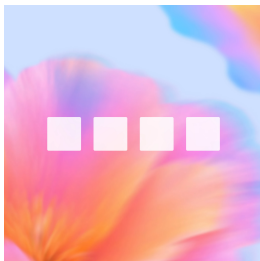
- [2026](#)

Author

OpenAI

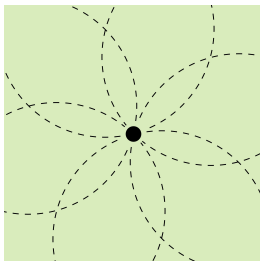
Keep reading

[View all](#)



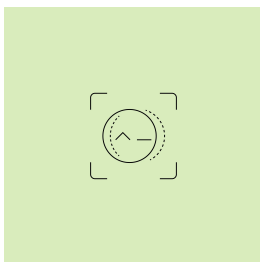
[GPT-5.5 System Card](#)

[Safety](#) Apr 23, 2026



[Inside our approach to the Model Spec](#)

[Research](#) Mar 25, 2026



[How we monitor internal coding agents for misalignment](#)

[Safety](#) Mar 19, 2026

[Daily article] May 16: Erik Campbell (Final Destination)

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 16 MAY 2026 · [SOURCE](#)

Erik Campbell is a character in Final Destination Bloodlines (2025), the sixth film in the supernatural horror franchise Final Destination, and was portrayed by Richard Harmon (pictured). Introduced as a grandson of Iris Campbell, who escaped Death in the 1960s, Erik is revealed to have been conceived out of his mother's affair and thus not in any danger. Despite this, while trying to help his brother cheat Death, Erik is sucked into an MRI machine by a wheelchair that crushes and impales him. The reveal of Erik's parentage was due to the film's crew wanting to subvert the audience's expectations regarding the order of deaths; a discarded idea involved twins. The directors were initially apprehensive about incorporating an MRI-machine death, but chose to include it due to positive feedback from the production team. Critics responded positively to Erik, describing him as a fan favorite and Bloodlines' best character, as well as praising his death scene and Harmon's performance. (Full article...).

Read more: https://en.wikipedia.org/wiki/Erik_Campbell_%28Final_Destination%29

Today's selected anniversaries:

1811:

Peninsular War: Allied British, Spanish, and Portuguese forces clashed with French troops at the Battle of Albuera (depicted) fought south of Badajoz, Spain. https://en.wikipedia.org/wiki/Battle_of_Albuera

1925:

The first modern performance of Claudio Monteverdi's opera Il ritorno d'Ulisse in patria occurred in Paris. https://en.wikipedia.org/wiki/Il_ritorno_d%27Ulisse_in_patria

1961:

Led by Park Chung Hee, the Military Revolution Committee carried out a bloodless coup against the government of Yun Po-sun in Seoul, ending the Second Republic of Korea. https://en.wikipedia.org/wiki/May_16_coup

2014:

At least 12 people were killed and 70 others injured when two bombs exploded in a market in Nairobi, Kenya. https://en.wikipedia.org/wiki/Gikomba_bombings

Wiktionary's word of the day:

sultanate: 1. A sovereign or vassal princely state—usually Muslim—where the ruler is styled sultan. 2. The office or position of a sultan. 3. About Word of the Day 4. Nominate a word 5. Leave feedback <https://en.wiktionary.org/wiki/sultanate>

Wikiquote quote of the day:

” --William H. Seward https://en.wikiquote.org/wiki/William_H._Seward